Partiel 1 d'Algorithmique

CPES ≪ Sciences des données, arts et cultures ≫

— L'examen dure deux heures. Il comporte 5 exercices, 4 théoriques, et le dernier est de la programmation.

— Veiller à indiquer votre nom sur l'ensemble des copies rendues, et à les numéroter le cas échéant.

— Cet examen est strictement personnel, toute tentative de triche sera sanctionnée.

— Les documents sont autorisés, mais limités à 5 feuilles recto-verso par personne.

— Aucun appareil numérique n'est autorisé : pas d'ordinateur, pas de téléphone, pas de montre connectée, etc. Pas de calculatrice non plus.

— Lorsqu'on demande l'analyse de complexité d'un algorithme, on vous demande d'argumenter pour démontrer quelle est la complexité de cet algorithme. Vous devez prouver la correction et la terminaison de l'algorithme seulement si cela vous est explicitement demandé.

— Pour l'exercice 5, vous devez écrire du code Python. Les erreurs mineures de syntaxe Python ne seront pas pénalisées, tant que le code écrit ressemble à un programme Python.

Exercice 1 : Notations asymptotiques

Donner les (meilleures) notations asymptotiques O des fonctions suivantes :

(a) $f(n) = 7 + 98n^3 + \frac{2}{13}n^4 + 54n =$

(b) $f(n) = 12n\sqrt{n} + 5n^2 + 12n\log^2(n) =$

(c) $f(n) = (\frac{5}{4})^n + n^3 + n\log(n) =$

Exercice 2 : Complexité récursive et théorème Maître

En appliquant le théorème Maître, donnez la complexité d'algo vérifiant les relations de récurrences suivantes : $T(n) = 3T(n/9) + O(\sqrt{n})$ et $T(n) = 2T(n/2) + O(\sqrt{n})$.

Exercice 3 : Algorithme glouton

On va regarder des intervalles semi-ouverts de la forme $]i, j]$. Par exemple, l'intervalle $]1, 3]$ contient tous les nombres x tel que $1 < x \leqslant 3$. Étant donné n intervalles semi-ouverts, on veut trouver un nombre minimum de points tel que chaque intervalle contient au moins un point. Donner deux critères gloutons pour résoudre ce problème. Appliquer chacun des deux critères à l'exemple suivant :

$$\mathcal{I} = \{]0, 2],]1, 4],]1, 4],]1.5, 4],]3, 5],]3, 5],]4, 6],]4, 7]\}$$

NB : aucune preuve d'optimalité n'est demandée.

Exercice 4 : Programmation Dynamique

Supposons que vous avez n marches d'escalier. Vous pouvez monter d'une, deux ou trois marches à la fois. Le problème est, étant donné un entier n, de calculer le nombre de façons différentes pour atteindre la n-ième marche. On note $T[n]$ ce nombre. Notez que, s'il y a par exemple 3 marches, monter d'abord 2 marches puis 1 et d'abord 1 marche puis 2, sont deux façons différentes d'atteindre la 3-ième marche.

Question 1 Donner la valeur de $T[i]$ pour $i = 1, 2, 3, 4, 5, 6$ (donnez juste la valeur, pas le détail des calculs).

Question 2 Proposer une stratégie de résolution du problème basée sur la programmation dynamique. Pour cette question, on vous demande seulement de donner la relation de récurrence (sans oublier les cas de bases).

Question 3 Écrire un pseudocode résolvant le problème et qui utilise le principe de mémoïsation. Votre algo doit prendre un entier n en entrée, et renvoyer la valeur de $T(n)$.

Question 4 Donner la complexité temporelle et spatiale de votre algo.

Exercice 5 : les coefficients binomiaux (Python)

Les coefficients binomiaux sont des coefficients qui permettent de calculer le nombre de sous-ensembles non-ordonnés à k éléments d'un ensemble de n éléments possibles. Ils sont utilisés dans de nombreux domaines mathématiques, et en particulier pour faire du *dénombrement*. Ils peuvent être écrits selon la formule suivante, avec k et n deux entiers naturels tels que $k \leqslant n$:

$$\binom{n}{k} = \frac{n \times (n-1) \times \cdots \times (n-k+1)}{k \times (k-1) \times \cdots \times 1} = \frac{n!}{k!(n-k)!}$$

Il est aisé de montrer que l'on peut exprimer un coefficient binomial de degré supérieur à partir de ceux de degrés inférieurs (c'est la *formule de Pascal*) :

$$\binom{n+1}{k+1} = \binom{n}{k} + \binom{n}{k+1}$$

Ceci ressemble beaucoup à une formule de récurrence, et on souhaite donc écrire un algorithme en Python calculant le coefficient binomial en (k, n) en exploitant cette relation. Il est alors naturel de faire appel à la programmation dynamique. En ré-indiçant l'expression ci-dessus et en ajoutant les cas de base adéquats, on obtient la formalisation suivante :

$$C(n,k) = \begin{cases} 1 & \text{si } k = 0 \\ 1 & \text{si } k = n \\ C(n-1,k-1) + C(n-1,k) & \text{sinon} \end{cases}$$

, où $C(n,k) = \binom{n}{k}$ pour simplifier les notations.

Question 1 Écrire une fonction Python `binomial_memoisation` prenant en paramètres les entiers naturels `n` et `k`, ainsi que la structure de données pour la mémoïsation `struct_memo`. Cette fonction devra retourner la valeur $C(n,k)$ **en utilisant la mémoïsation** (stratégie *top-down*, incluant donc de la récursion).

Question 2 Écrire une fonction Python `binomial_sans_memoisation`, prenant en paramètre les entiers naturels `n` et `k`. Cette fonction devra retourner la valeur $C(n,k)$ **sans utiliser la mémoïsation** (stratégie *bottom-up*, via un procédé itératif).

Question 3 Déterminer les complexités temporelle et spatiale de l'une des deux fonctions écrites de votre choix. Justifier.